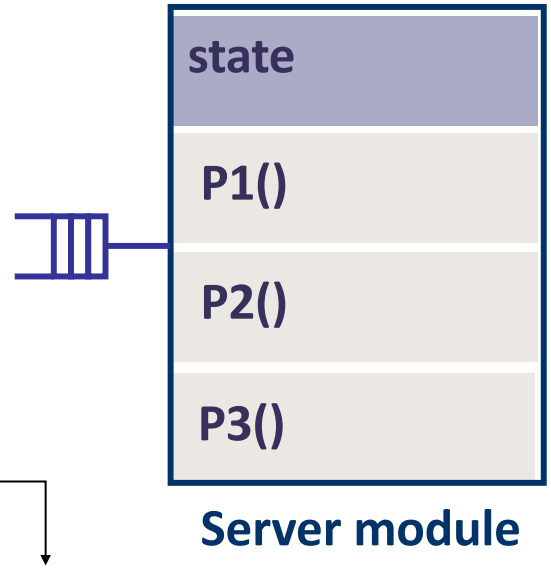
CompSci 390-03-26S

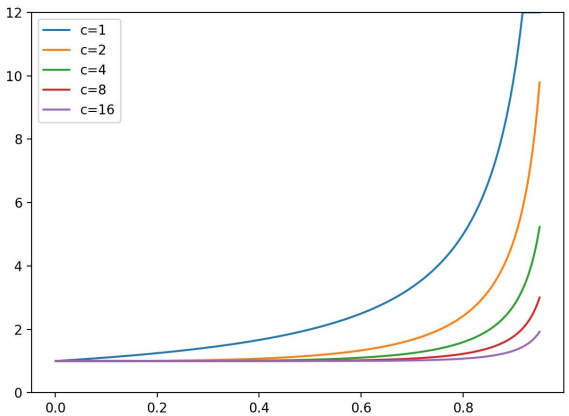
Distributed and Networked Systems

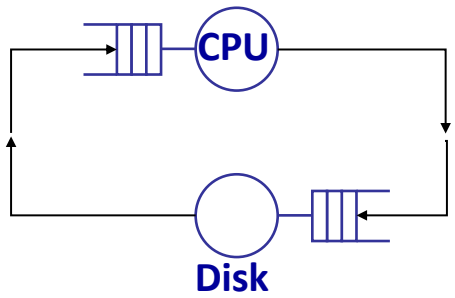
Services and State

The story so far: servers and their workloads

- Servers handle requests from clients
- Requests are RPCs: call method of service API
- Performance depends on workload and service implementation: e.g., λ, D in our models.
- How to improve performance for a given λ, D?







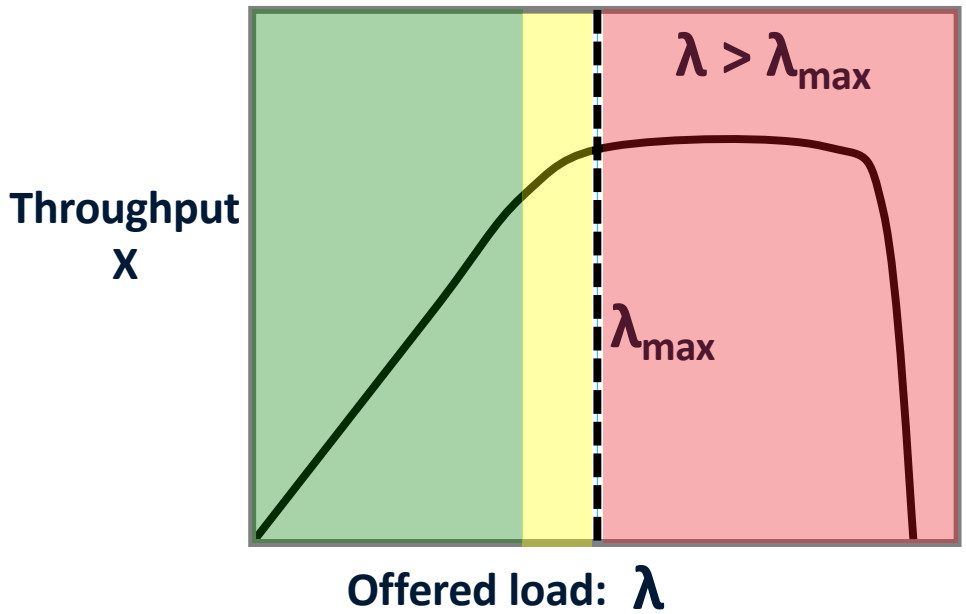
Overload

What happens when a server is overloaded?

- **Overload**: close to saturation.

→ queues grow without bound

- What happens to:
  - Utilization?
  - Memory consumption?
  - Response time?
  - Throughput?
- What can we do about it?

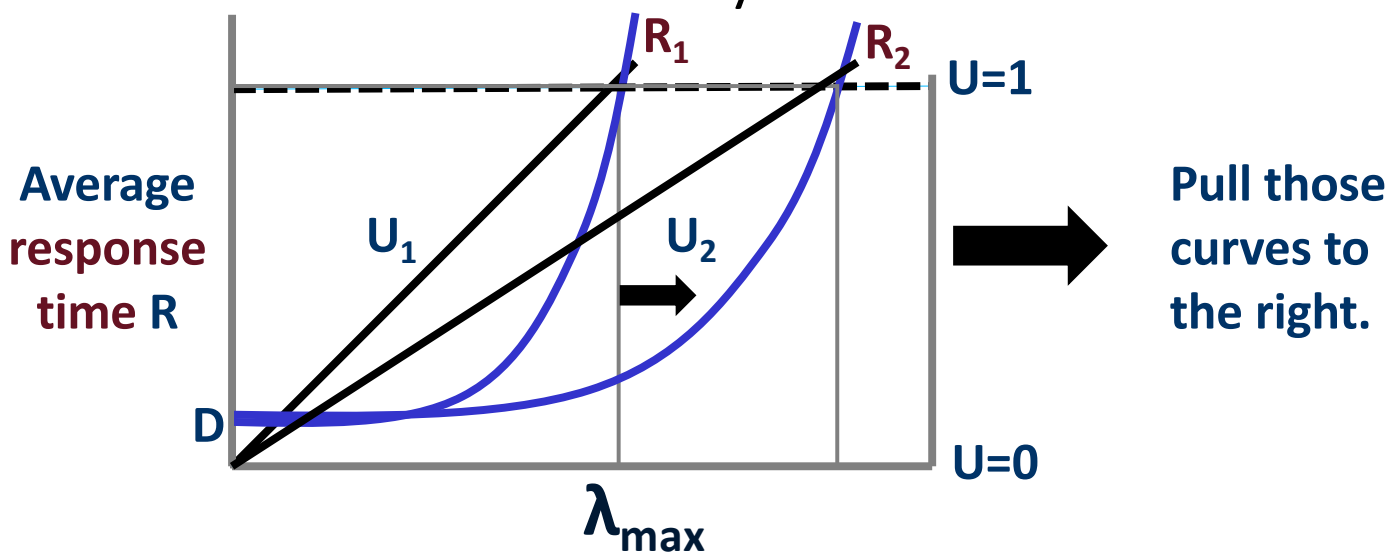


Overload: options

1. **Choke**. Hope that things get better.
   - They might! But hope is not a plan. Make a plan.
2. **Scale up**. Add more resources, e.g., more cores.
   - What are the limits to scaling?
3. **Admission control**. Drop selected requests.
   - How to select? **Load conditioning**
4. **Scale out**. Spread the load across more servers.
   - **Horizontal** scaling
   - Also called: **elastic scaling**

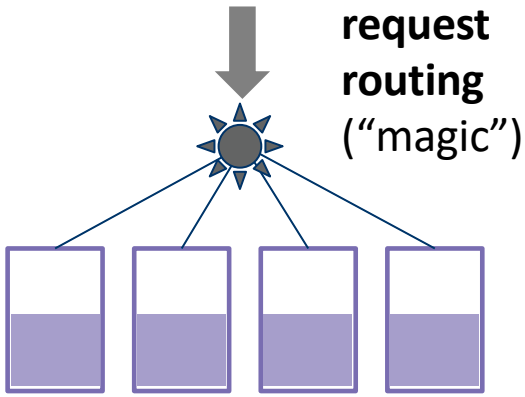
Scaling

- Effective scaling—by any means—pulls curves "right".
- Handle higher load (λ_{max}) with (mean) $R < R_{max}$.
- **Examples**: faster cores → lower D; faster I/O → less waiting; more cores → more concurrency.

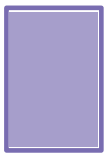


Service: A key abstraction for a cloud OS

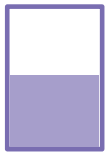
- A **service** is a group/set of servers working together to serve an API.
- Deploy/manage as a unit, e.g., callers see a single logical service endpoint.
  - E.g, DNS name or Virtual IP address (VIP)
- **Elastic**. Add/remove servers to "right-size" for the request load (e.g., to target U or R)
- Design questions, lots of options:
  - Which server does a request go to?
  - Who decides?
  - How is the choice applied?



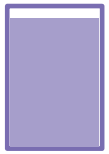
Scaling and performance: example



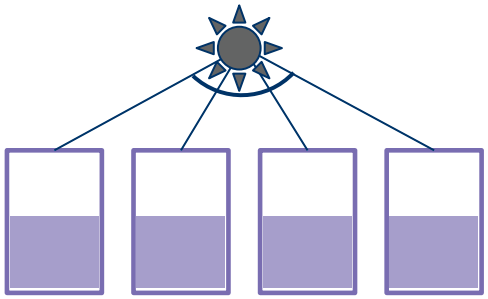
Saturated: peak throughput λ = 100 r/s



λ = 50 r/s, R = 10 ms
U?

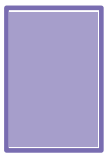


λ = 90 r/s
U? R?

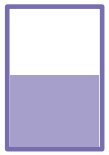


4x U=50%
Peak λ? R?

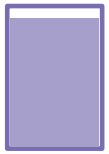
Scaling and performance: example



Saturated: peak throughput λ = 100 r/s: λ_{max}

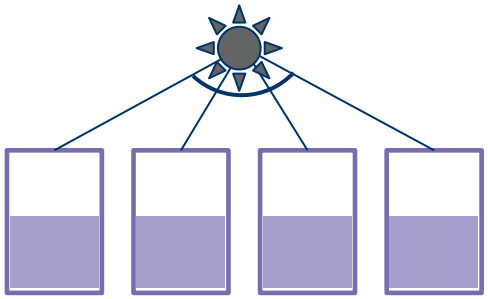


λ = 50 r/s, R = 10 ms
U = 50% (λ/ λ_{max})



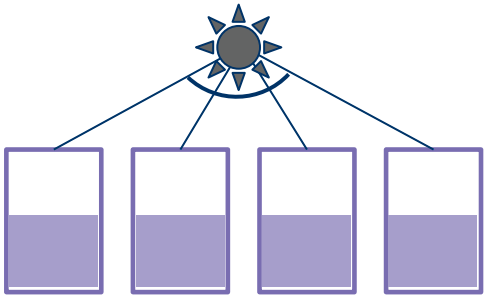
λ = 90 r/s
U = 90%, R = 50 ms

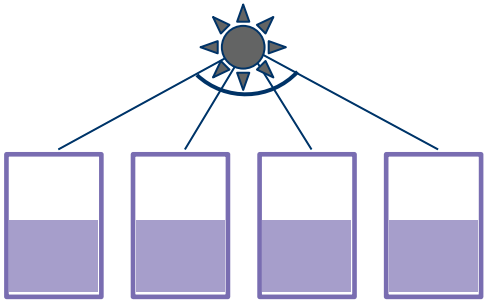
Note: these are rough guesses from "back of napkin" analysis.



4x U=50%
R = 50 ms
Peak λ = 400 r/s

Two approaches to request routing (load spreading)



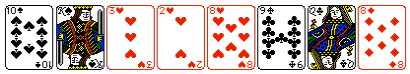


Option 1: **virtual IP** (VIP), "L4"

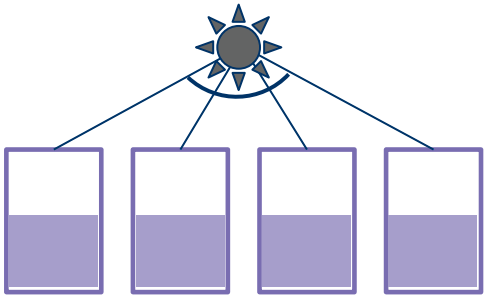
- Round-robin by connections
- Route on TCP/IP 5-tuple
- Protocol+{source IP/port, dest IP/port}
- Uniquely identifies connection
- Route by **proxy**: L4 load balancer

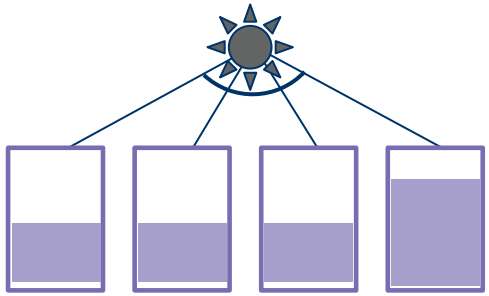
Option 2: "L7", e.g., data **sharding**

- Route on request attributes
- E.g., target object ID, HTTP cookie
- Proxy? A little more involved
- Commonly "proxyless"



Two approaches to request routing (load spreading)



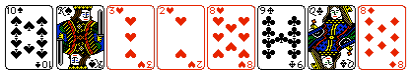


Option 1: **virtual IP** (VIP), L4

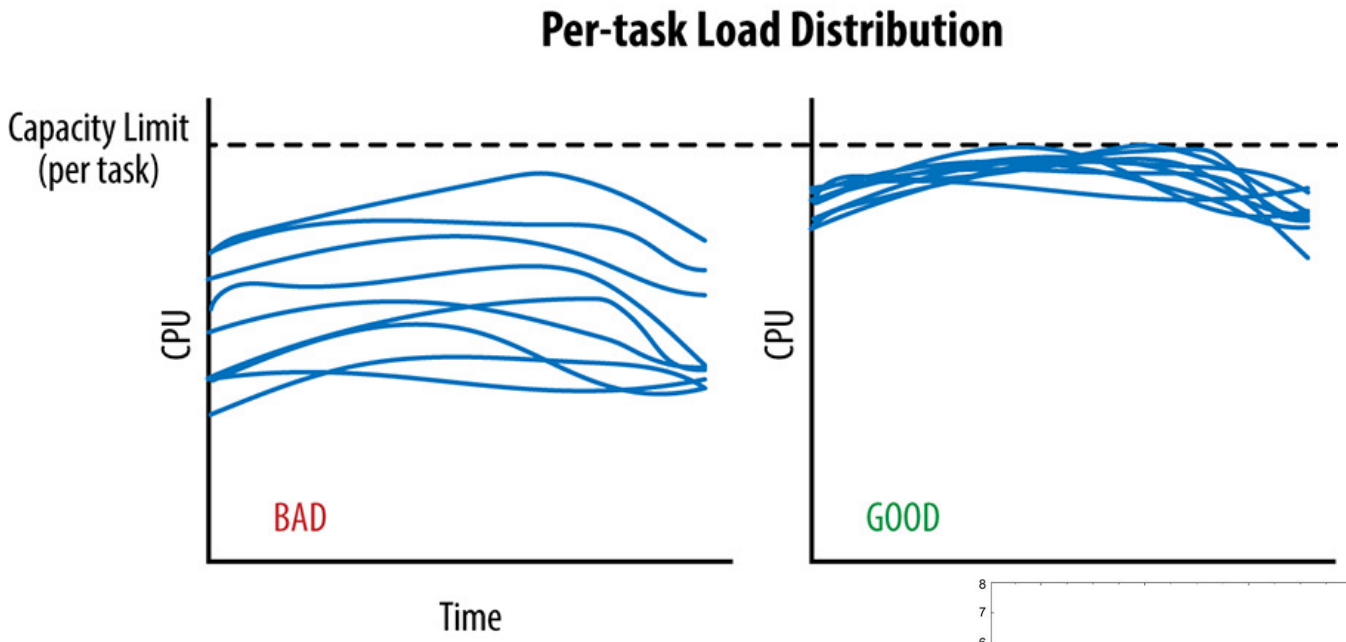
- Round-robin by connections
- Statistically likely to balance load

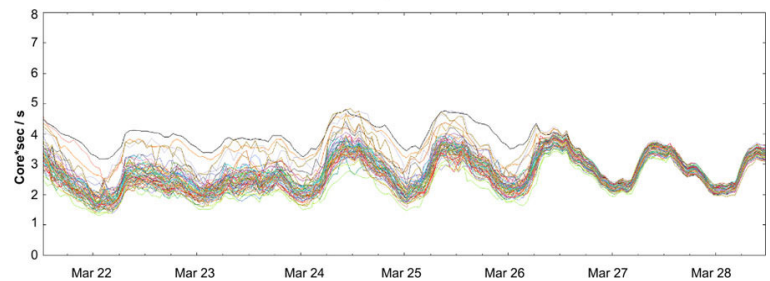
Option 2: L7, e.g., data **sharding**

- Less likely to balance load evenly!
- E.g., for "hot" objects
- Skewed distributions are common
- What does it do to response time?



Load balancing

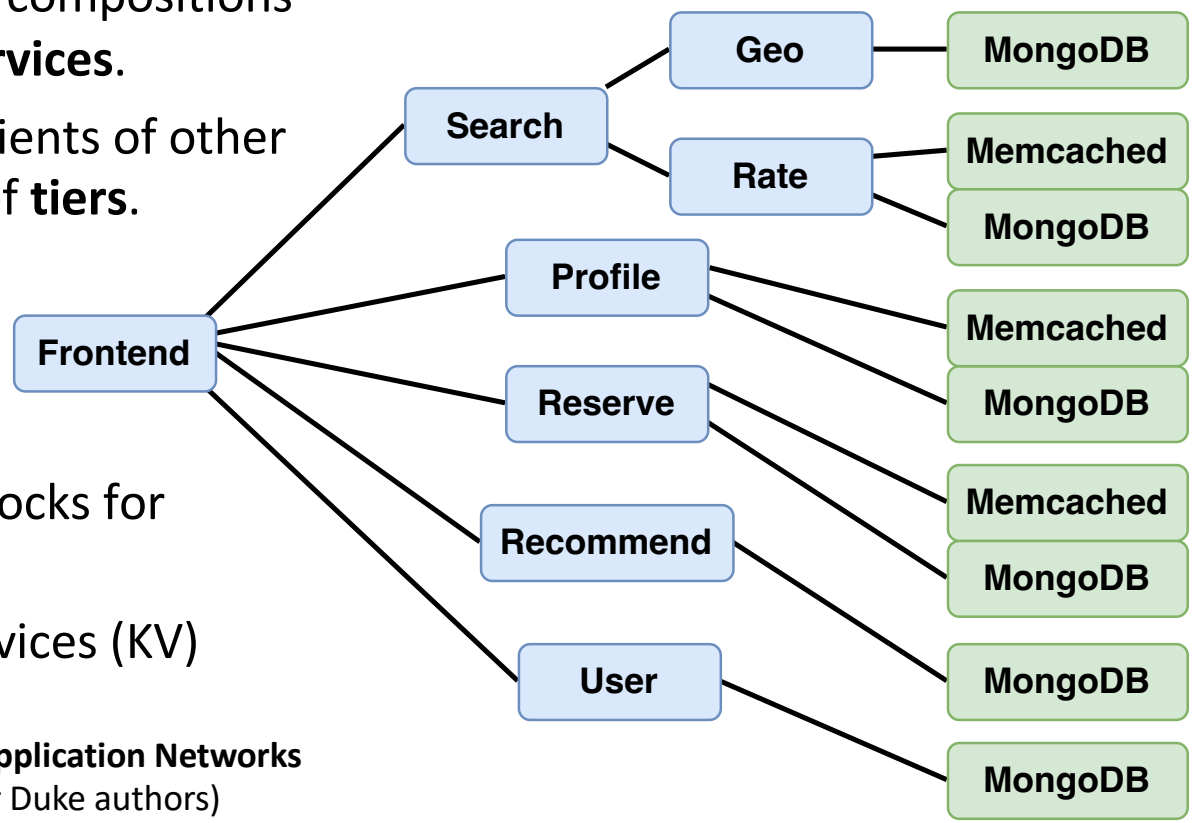




https://landing.google.com/sre/sre-book/chapters/load-balancing-datacenter/

Service composition: microservices

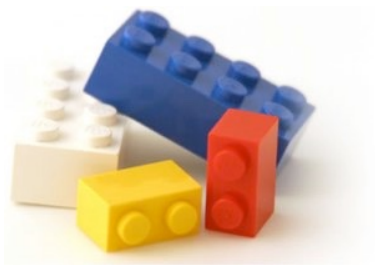
- Today's services are compositions of modular **microservices**.
- Services are often clients of other services: we speak of **tiers**.



- Reusable building blocks for common needs
- E.g., storage tier services (KV)

From **High-level Programming for Application Networks**
NSDI 2025 (Danyang Zhou and other Duke authors)

Microservices



DeathstarBench

Yu Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinsky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In *Proceedings of 2019 Architectural Support for Programming Languages and Operating Systems (ASPLOS'19)*. ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3297858.3304013

Twitter, Netflix, Apple, and EBay having already adopted the microservices application model [6, 18, 19], and Netflix reporting more than 200 unique microservices in their ecosystem, as of the end of 2016 [18, 19].

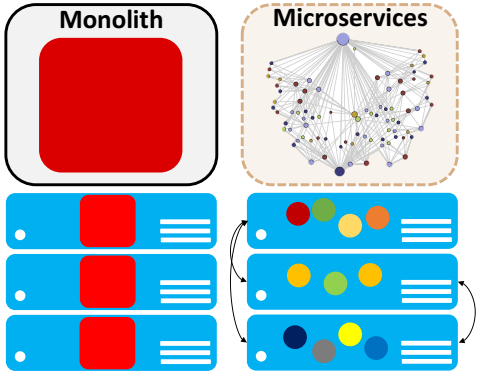


Figure 1. Differences in the deployment of monoliths and microservices.

The increasing popularity of microservices is justified by several reasons. First, they promote composable software design, simplifying and accelerating development, with each microservice being responsible for a small subset of the application's functionality. The richer the functionality of cloud services becomes, the more the modular design of microservices helps manage system complexity. They similarly facilitate deploying, scaling, and updating individual microservices independently, avoiding long development cycles, and improving elasticity. Fig. 1 shows the deployment differences

Buzzword: service mesh

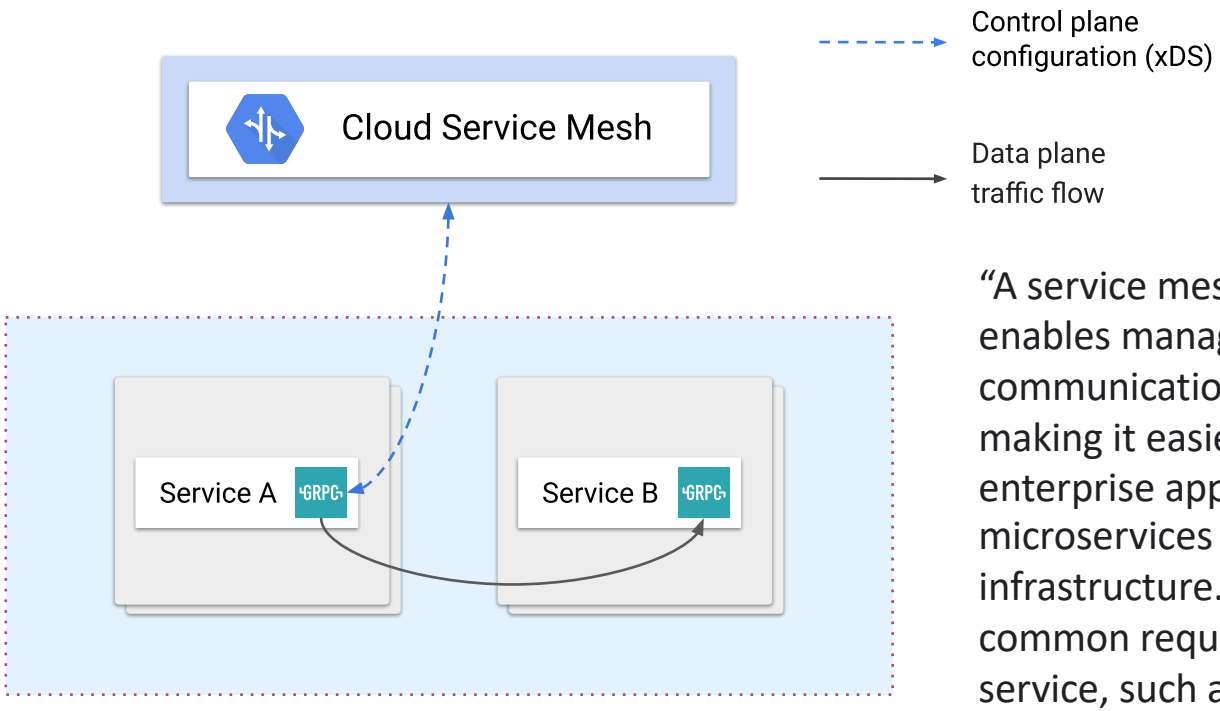
Open Service Mesh

envoy

Google Cloud Anthos

Istio

Cloud service mesh



Service mesh enabled by
proxyless gRPC services

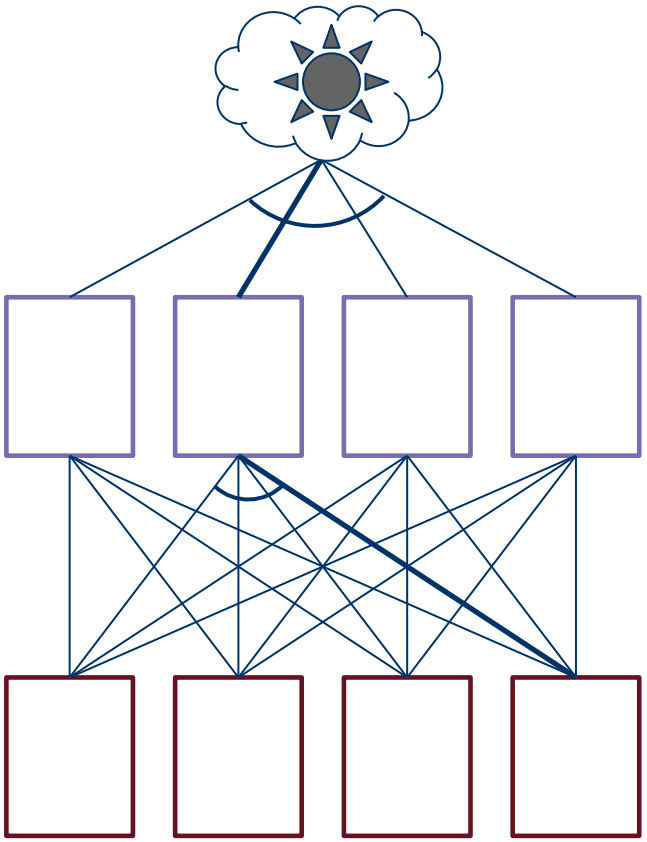
"A service mesh is an architecture that enables managed, observable, and secure communication among your services, making it easier for you to create robust enterprise applications made up of many microservices on your chosen infrastructure. Service meshes manage the common requirements of running a service, such as monitoring, networking, and security...and traffic management."

See https://docs.cloud.google.com/service-mesh/docs/overview

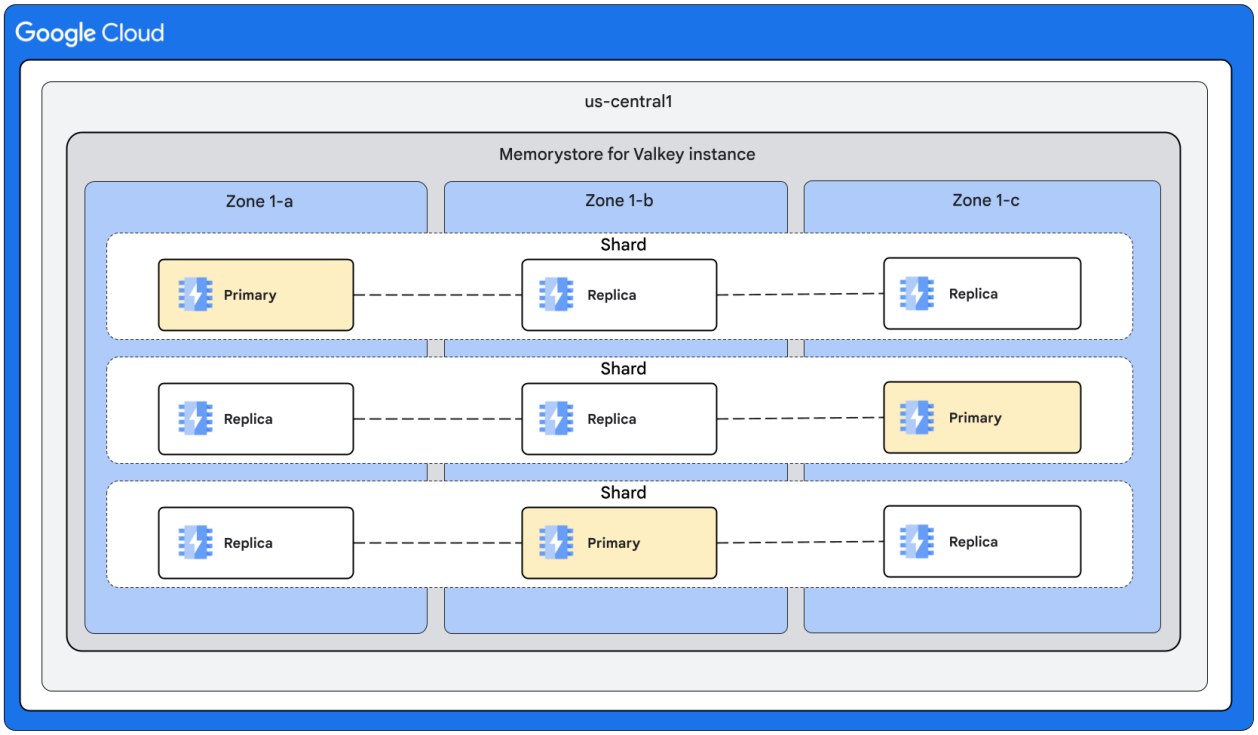
Google Cloud

Typical structure

- Front-end tier: "stateless"
- Stateful storage tier
- Cloud routing to front-end by VIP
- Proxyless routing to sharded storage back-ends
- E.g., managed by cloud provider



Google Cloud: Memorystore for Valkey



https://docs.cloud.google.com/memorystore/docs/valkey

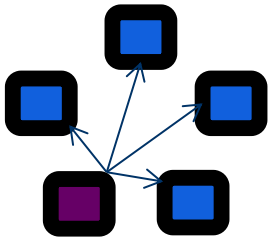
What about failures?

- **Nodes fail. Servers fail**. Let's make some assumptions:
  - **Fail-stop** or **fail-fast** fault model
  - Either function correctly **or** go silent
  - Either restart correctly **or** stay silent
  - A restarted node loses its memory state, and recovers its secondary/nonvolatile (disk) state, or state on storage tier.
  - Alternative: **byzantine** nodes may lie or be unfaithful (later).

- If failures are random/independent, the probability of **some** failure is linear with the number of units.
  - Higher scale → less reliable?

Replication

- Option**: replicate** the server functions and state.
  - Run multiple copies (**replicas**).
  - Each has the same state.
  - They comprise a **replica group**.
  - One replica is **primary** or **leader**.
  - The others are **backup** replicas.
  - (Or standby or secondary or **follower**.)



- Replication → highly **available** services: "always up".

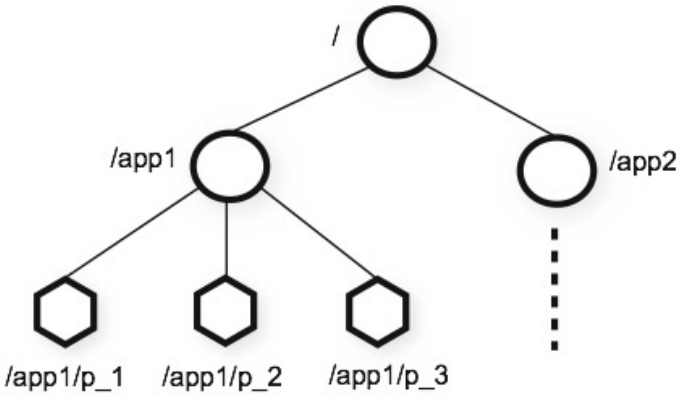
Group coordination

- Group servers into services for scale and resilience
- Now we need to coordinate them!
  - Service discovery: finding and binding service endpoints by name
  - Which servers are members of the group?
  - Which server is the leader?
  - Which servers own which shards?
  - Failure detection
  - Locking to distribute data and functions
  - Change notification
- We need a **coordination service**

ZooKeeper: a coordination service

- Hierarchical namespace
- Znodes: data and/or named children
- Znode contents: a blob of bytes (data)
- Set **watch** on node to notify of changes
- Modeled on Google's Chubby, see also etcd

- *create* : creates a node in a parent path
- *delete* : deletes a node
- *exists* : tests if a node exists at a path
- *get data* : reads the data from a node
- *set data* : writes data to a node
- *get children* : retrieves {children} of a node

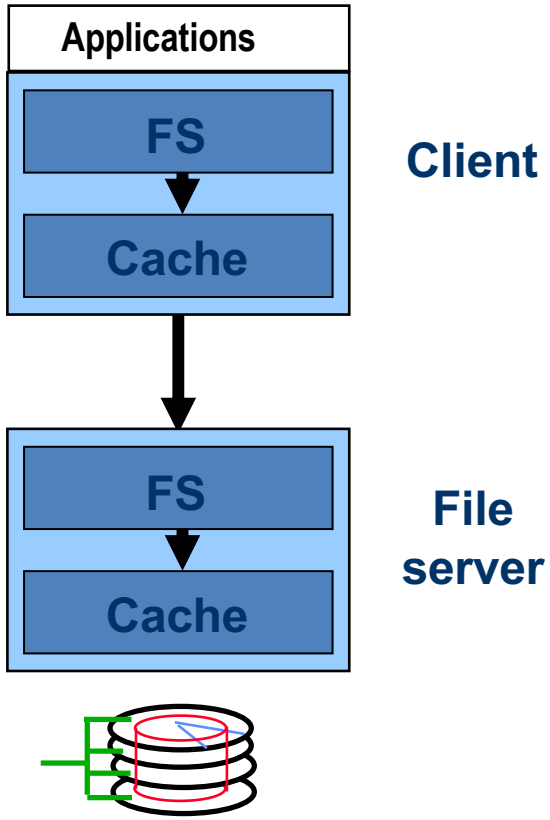


Coordination as a file service

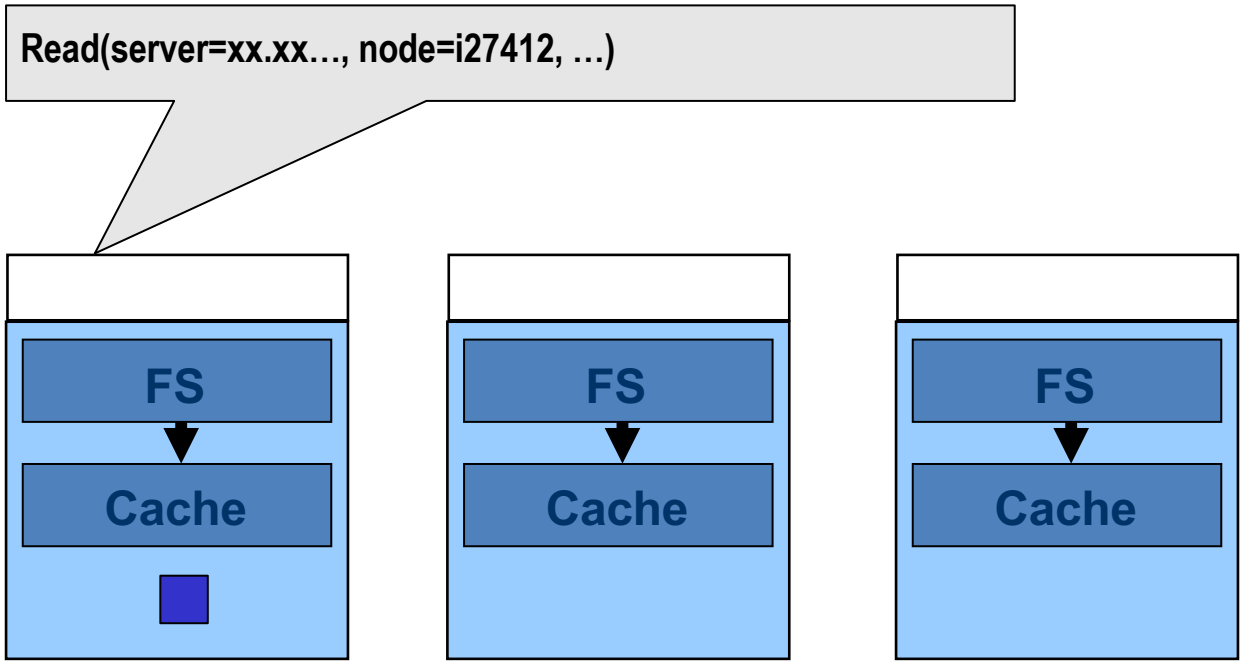
- Chubby etc. are **strongly consistent** file systems. Clients may:
  - **cache** files→caches tracked by server
  - create **ephemeral** file to "own" a name as long as the client lives
  - subscribe to receive change notifications on files (**watches**)
  - lock/unlock files

```
initialize()
create/open ephemeral file /ls/cell/app/$self
to say "I am alive"
try to acquire /ls/cell/app/leader
if (succeed)
  beLeaderForApp()
else
  beWorkerForApp()
```

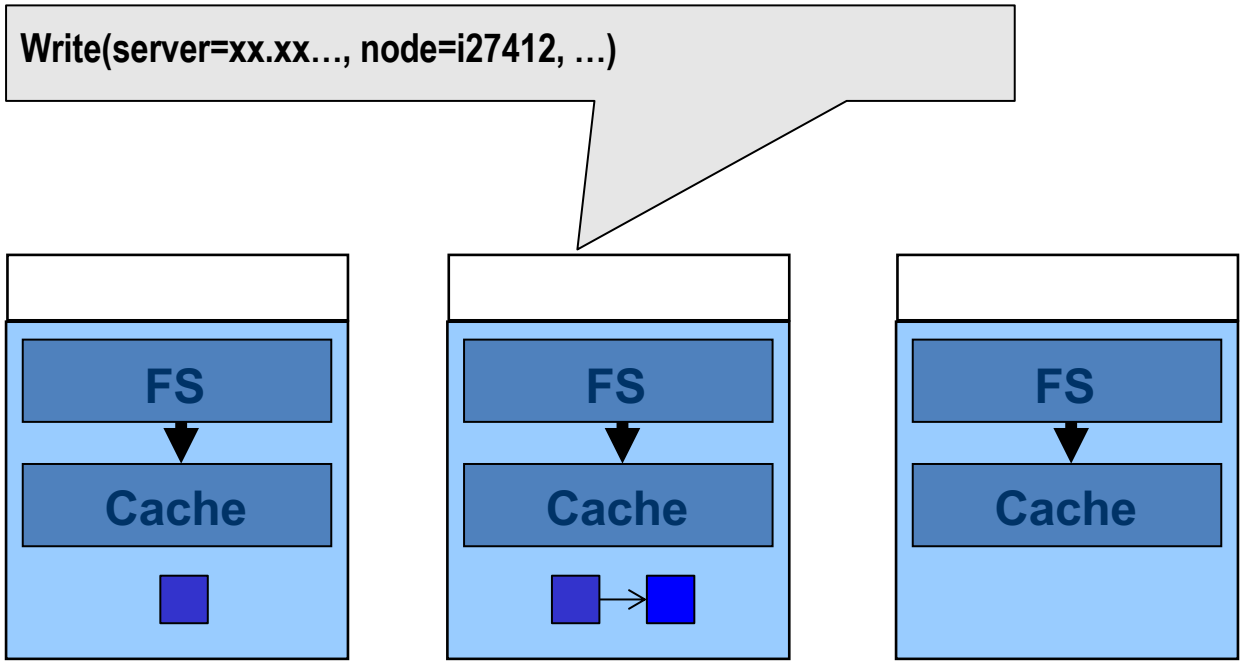
Caching in a file service



Caching with multiple clients

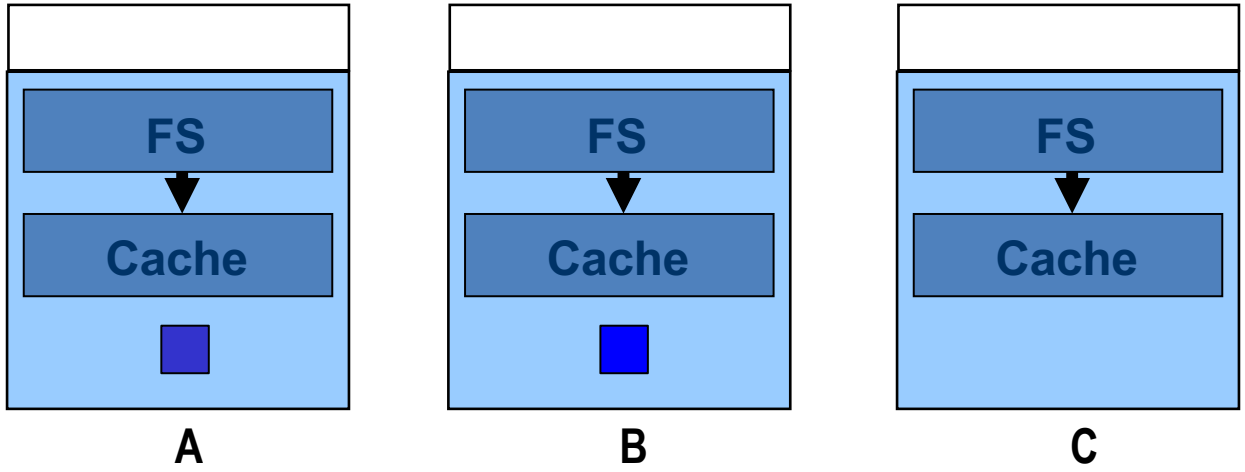


Caching with multiple clients

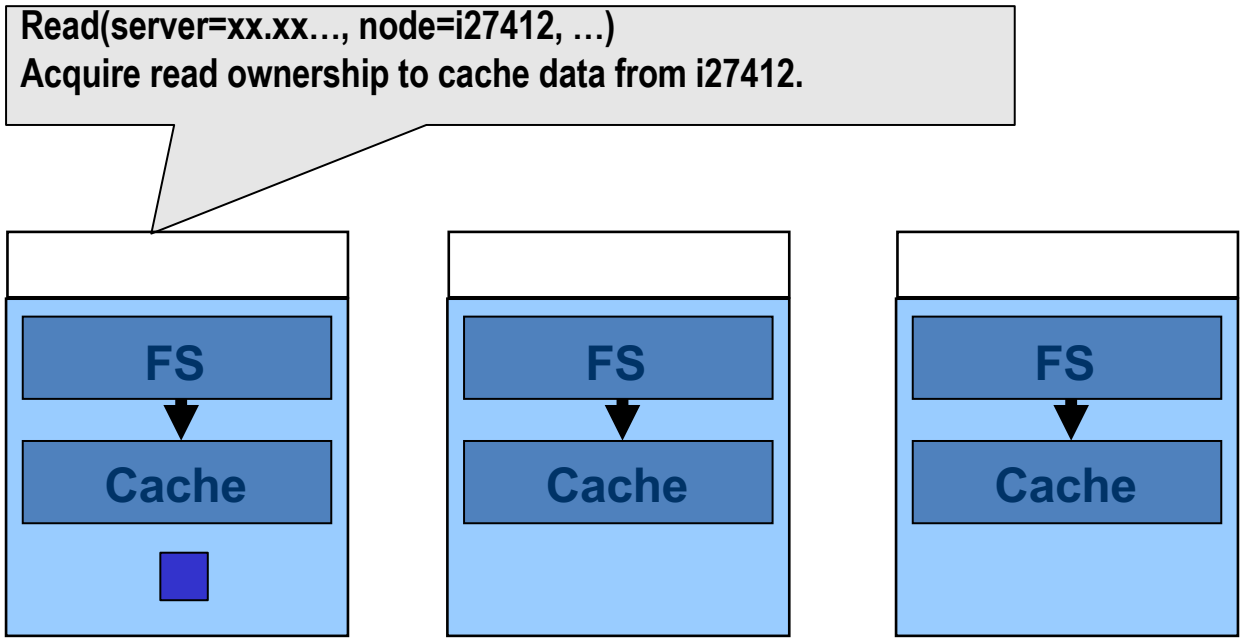


Caching with multiple clients

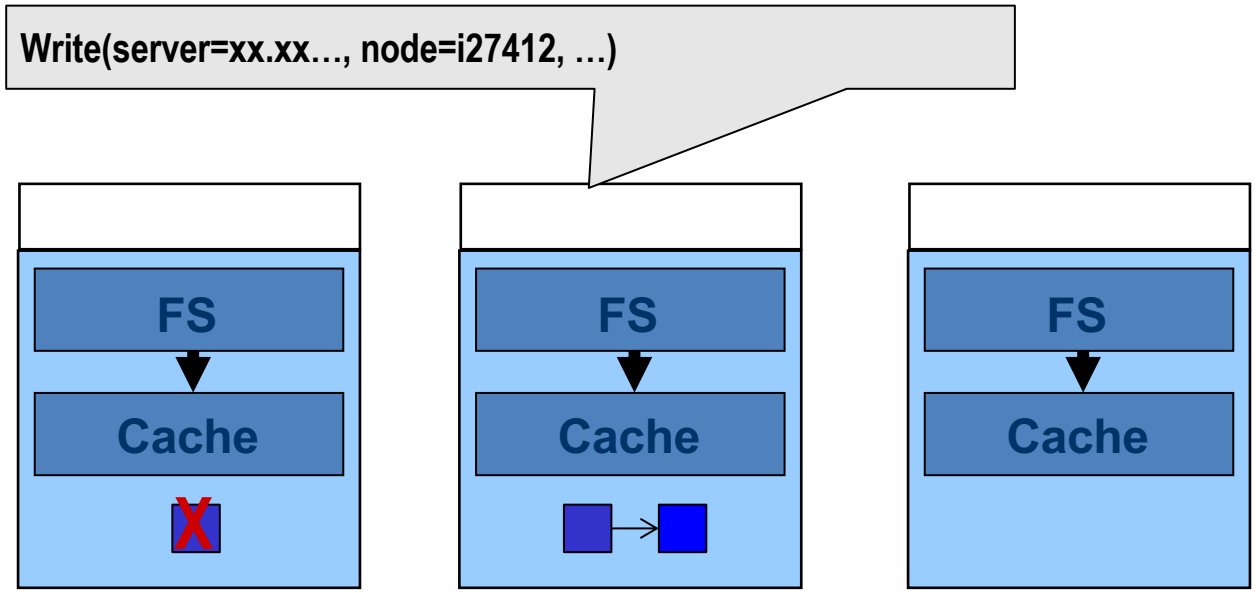
- If client A reads that file, what data does it return?
- What is the "right" answer?
- How to keep the copies "in sync"?



Strong consistency with read-write ownership

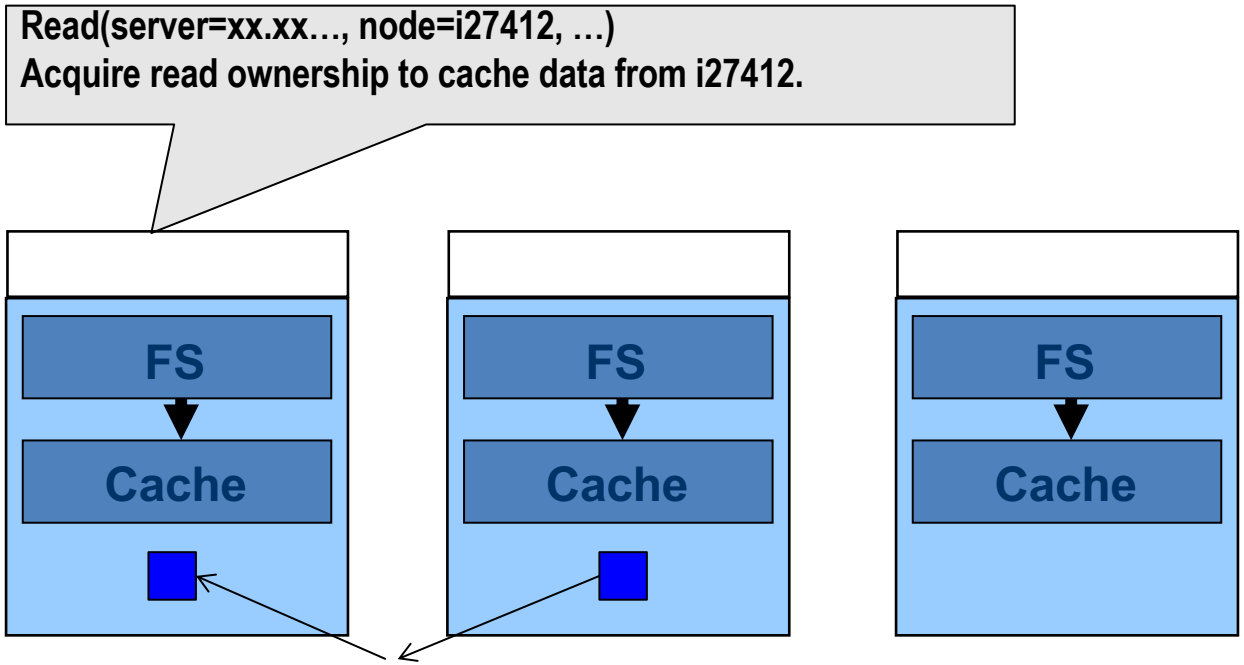


Strong consistency with read-write ownership



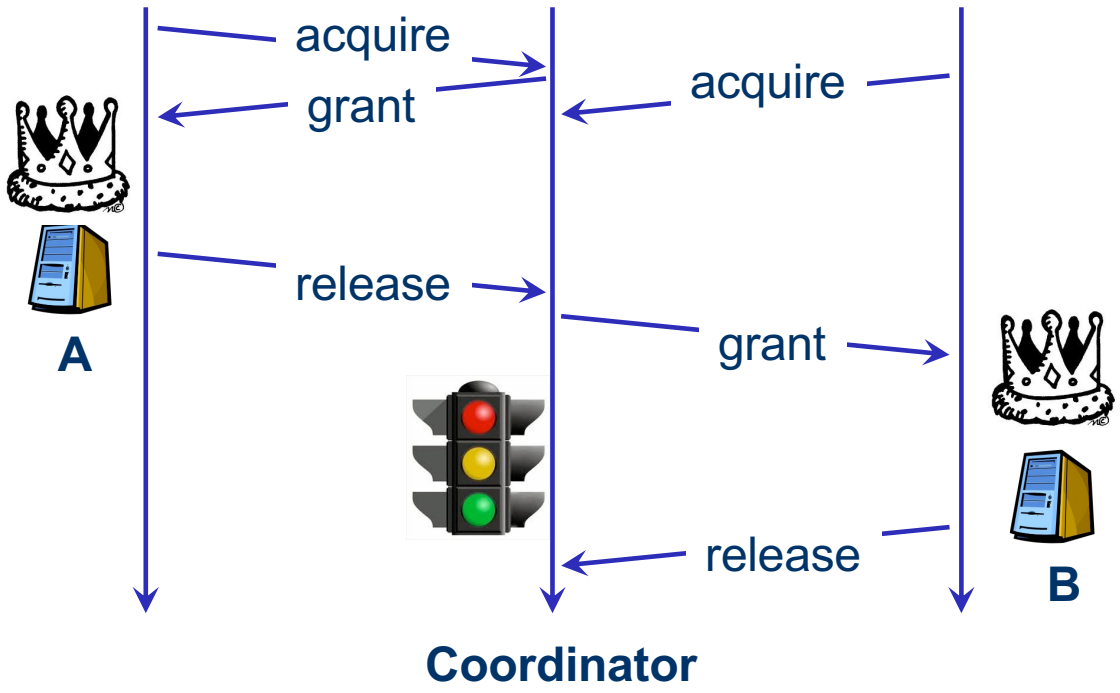
Writer must acquire write ownership for i27412.
Server revokes reader rights before granting write ownership
Clients relinquish read rights and invalidate cached content for i27412.

Strong consistency with read-write ownership

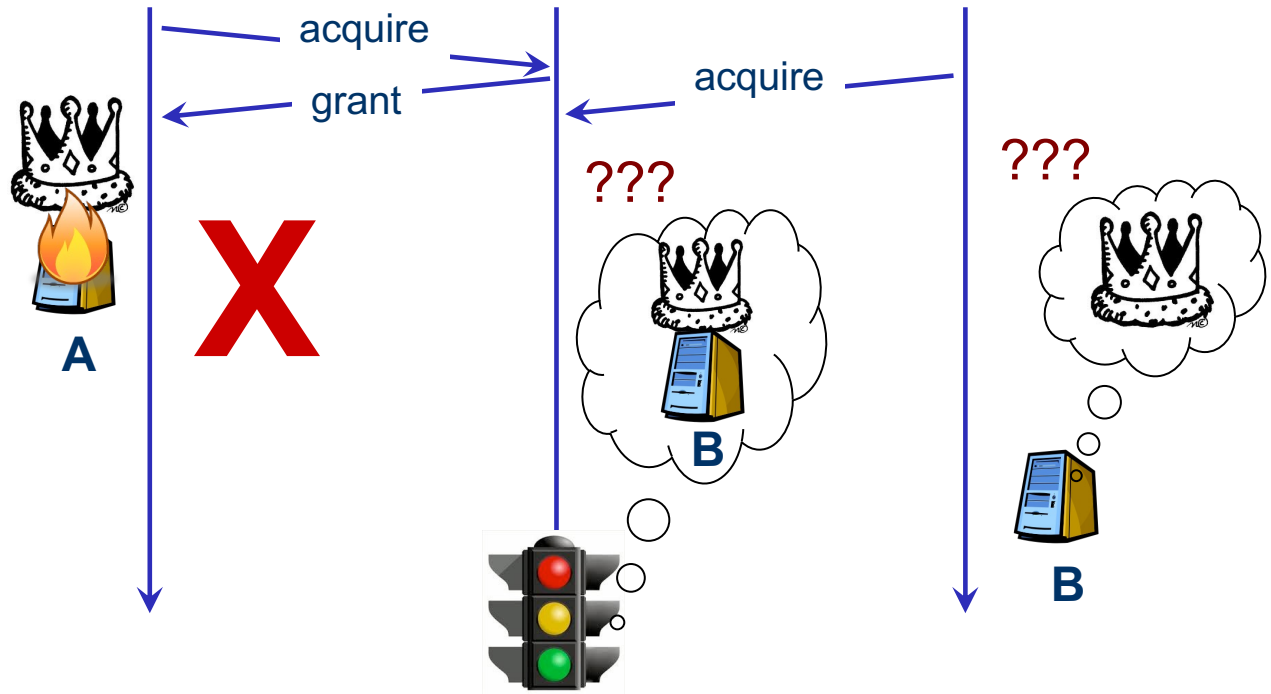


Server recalls write ownership (if any) before granting.
Writer pushes modified data for i27412 and relinquishes.

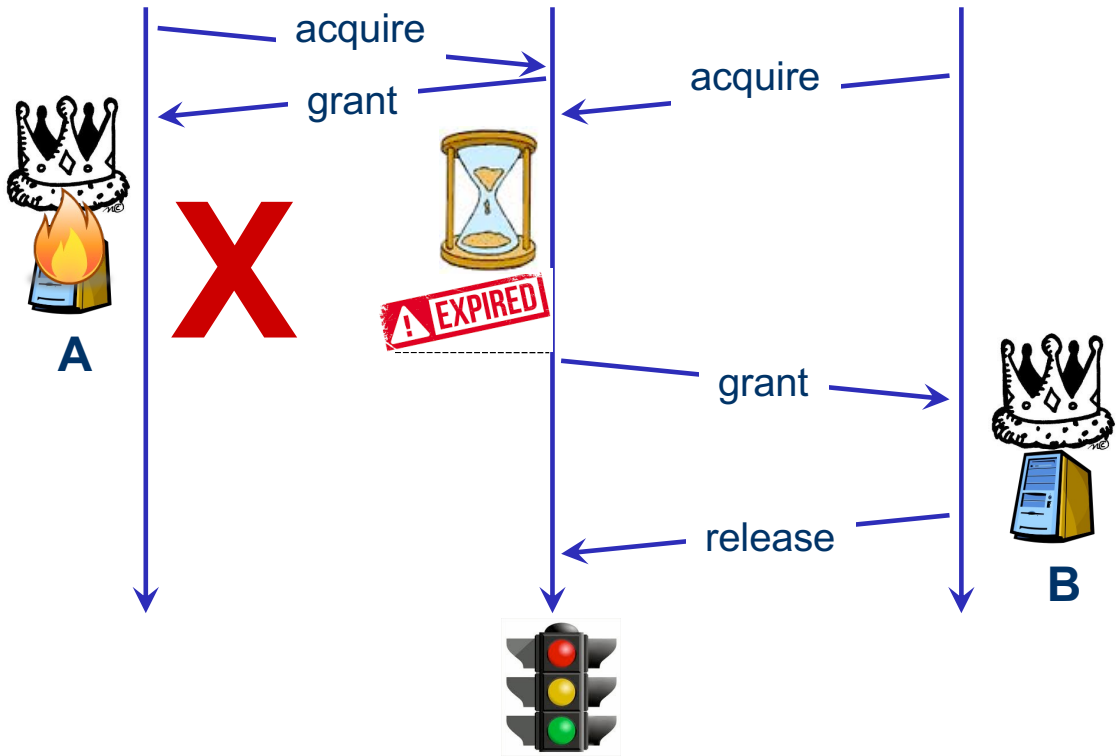
Lock service: normal operation



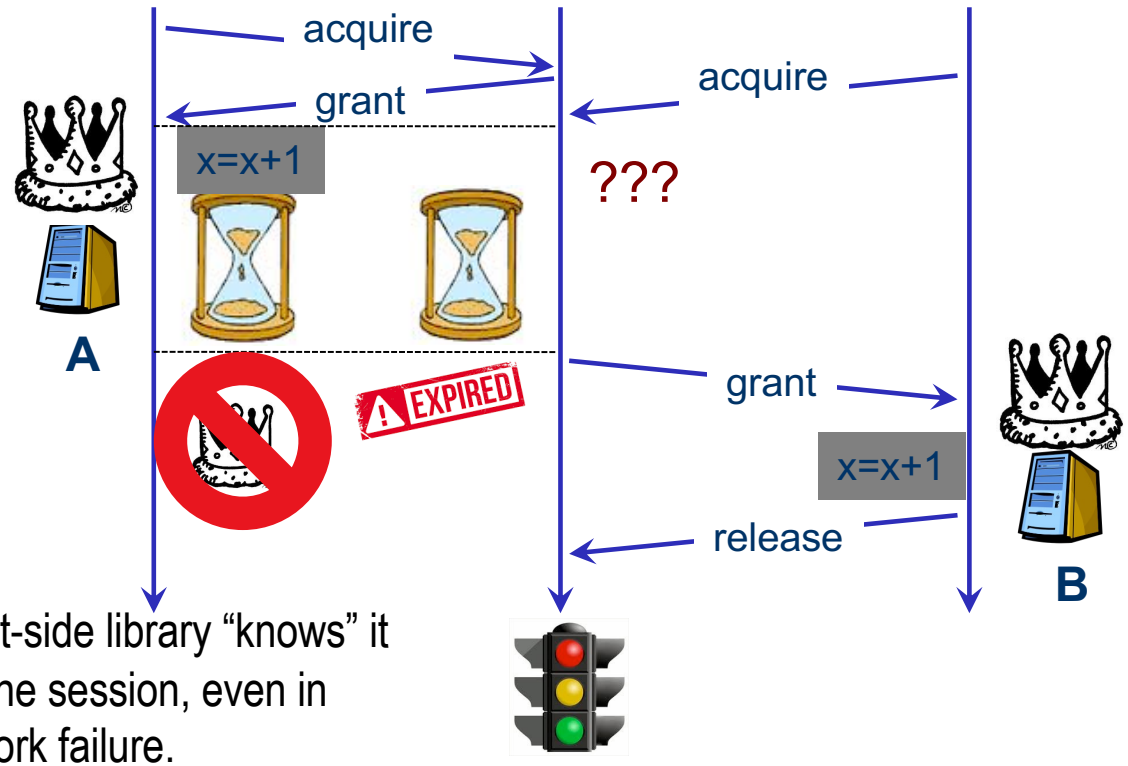
A lock service in the real world



Using leases to track client sessions

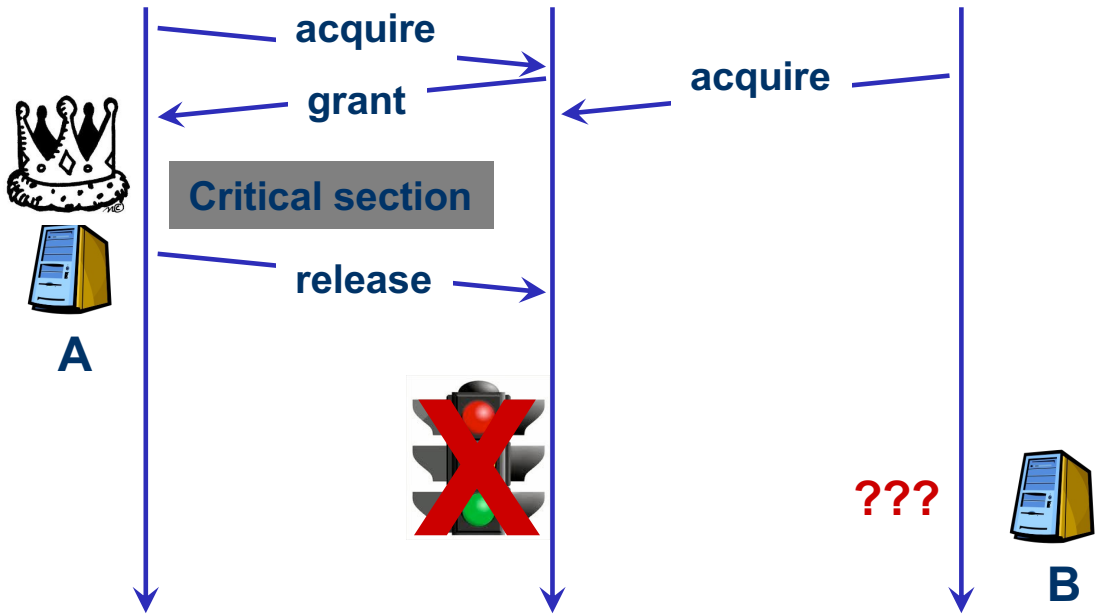


Never two kings at once



Client-side library "knows" it lost the session, even in network failure.

Also consider lock server failure…



Coordinator functions

Allow app servers to:

- claim roles, functions, or data, with mutual exclusion
- advertise their liveness and roles to peers
- share configuration data with strong consistency
  - e.g., service endpoint addresses, roles
- coordinate their accesses to other services (r/w locking)

Notify app servers if/when:

- configuration changes occur elsewhere in the application
- consensus properties cannot be assured due session loss (“jeopardy”)

Microservices

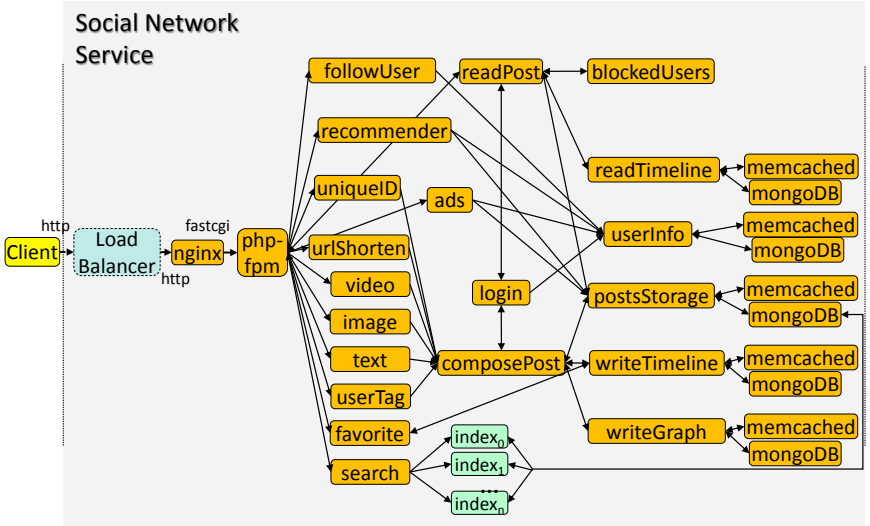


Figure 4. The architecture (microservices dependency graph) of *Social Network*.

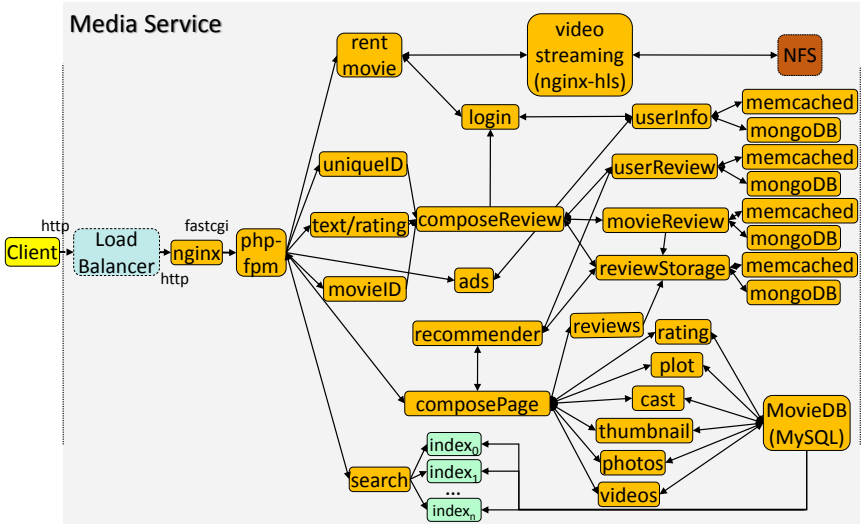
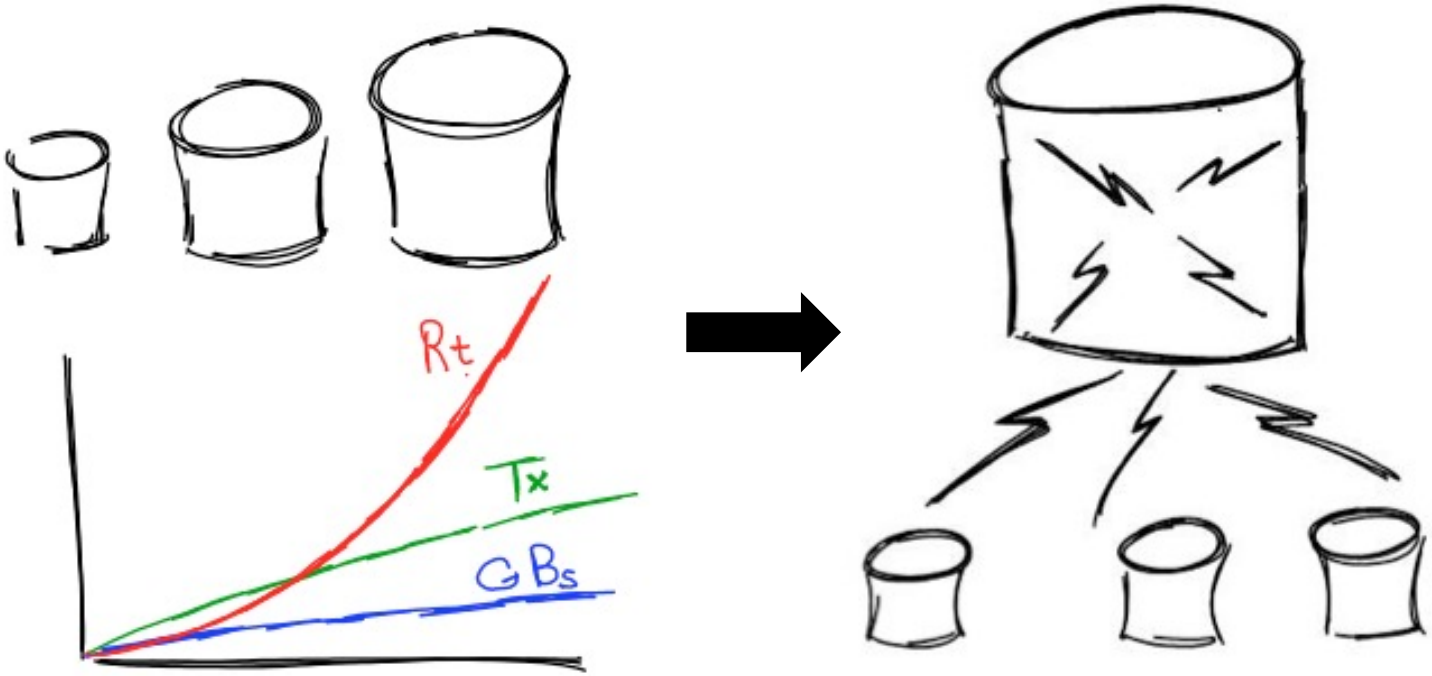


Figure 5. The architecture of the *Media Service* for reviewing, renting, and streaming movies.

Yu Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinsky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In *Proceedings of 2019 Architectural Support for Programming Languages and Operating Systems (ASPLOS'19)*. ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3297858.3304013

Scale-up vs. scale-out



"We're gonna need a bigger server." **"Get more and lash them together."**

Borrowed years ago from http://dbshards.com/dbshards/database-sharding-white-paper/